Predicting Student Pass/Fail Outcomes with Decision Tree and Random Forest on the UCI Student Performance Dataset

Project title	Predicting Student Pass/Fail Outcomes with Decision Tree and Random Forest on the UCI Student Performance Dataset
Student name(s)	Azizullah Ali Yar And Abozar Shakori
Student ID(s)	Not provided
Department / class	Information System, Semester 8
Course name	Data Mining
Instructor name	Dr. Ziaullah Momand
Submission date	June 18, 2026
Project type	Classification

Project Summary

Problem addressed	Predict whether a student is likely to pass or fail by using previous grades and background attributes.
Dataset used	UCI Student Performance Dataset; 649 student records and 33 original columns, including demographic, school, social, and grade variables.
Data mining task	Supervised binary classification. The model learns from labeled examples where Pass = 1 if G3 >= 10 and Fail = 0 if G3 < 10.
Main algorithm(s)	Decision Tree Classifier as an interpretable baseline, and Random Forest Classifier as an ensemble model for stronger accuracy.
Best result	Random Forest achieved 90.77% accuracy on the 20% test set, compared with 88.46% for Decision Tree.
Main conclusion	Earlier grades, especially G2 and G1, were the strongest predictors. Random Forest performed best because it combines many trees instead of relying on one tree.

1. Introduction

Student performance prediction is an educational data mining task that uses historical student data to estimate future academic outcomes. In this project, the outcome is not an exact final score; it is a clear pass/fail class. This makes the task easier to explain and suitable for classification algorithms.

The specific problem is to predict whether a student will pass or fail by using the UCI Student Performance Dataset. The model uses input features such as first-period grade, second-period grade, study time, absences, family support, school support, and other student-related attributes.

This project is important because a prediction model can help identify students who may need academic support. The purpose is not to replace teachers or make final decisions automatically; the purpose is to show how data mining methods can find useful patterns from educational data.

The objectives of the project are:

- Load the UCI Student Performance Dataset in Google Colab.
- Define a binary target variable named pass_fail from the final grade G3.
- Clean, encode, and split the dataset into training and testing parts.
- Train a Decision Tree model to create an interpretable baseline.
- Train a Random Forest model to test whether an ensemble method improves the result.
- Evaluate both models using accuracy, precision, recall, F1-score, and confusion matrix.
- Interpret feature importance to understand which variables influenced prediction most.

The expected output is a trained classification model, a comparison of model performance, and an explanation of the most important features used in the prediction.

2. Dataset Description

The dataset used in this project is the UCI Student Performance Dataset. It contains student achievement data from secondary education in Portuguese schools. The data includes academic grades, demographic information, family background, study behavior, school support, and social factors.

The final grade column G3 is the original output variable. For this project, G3 was converted into a binary classification target named pass_fail. A student was labeled Pass when G3 was 10 or more, and Fail when G3 was below 10.

Dataset name	Student Performance Dataset
Dataset source	UCI Machine Learning Repository
Number of records	649
Original columns	33
Target variable	G3 final grade converted to pass_fail
Prediction type	Binary classification: Pass or Fail
Subject used	Student performance data from the UCI repository

Important Attributes

Attribute	Meaning
G1	First period grade, numeric from 0 to 20.
G2	Second period grade, numeric from 0 to 20.
G3	Final grade, numeric from 0 to 20; used to create the target.
studytime	Weekly study time category; higher value means more study hours.
failures	Number of past class failures; useful for identifying academic risk.
absences	Number of school absences.
freetime / goout	Social and lifestyle-related attributes.
school, sex, address, family variables	Categorical demographic and school-related features.

A key data quality note is that G1 and G2 are strongly related to G3 because they are earlier grade periods from the same course. Keeping G1 and G2 makes prediction more accurate, but it also means the model is using information that is already close to the final result.

The dataset is public and anonymized, so it does not include direct student names. However, if this type of model were used in a real school, fairness and privacy would be important concerns. Predictions should support students rather than label or punish them.

3. Data Preprocessing

Data preprocessing converted the raw dataset into a format that classification algorithms can use. The original data contained a mix of numeric columns, such as grades and absences, and categorical columns, such as school, sex, address, support status, and family information.

First, the dataset was checked for missing values using df.isnull().sum(). No missing values were found, so no imputation or row deletion was required.

Step	Action taken
Missing values	Checked with isnull().sum(); total missing values were 0.
Target creation	Created pass_fail from G3: 1 when G3 >= 10 and 0 when G3 < 10.
Target leakage handling	Dropped G3 from X because it directly defines the answer. Keeping it would make the model unfairly easy.
Feature/target split	X contained all input columns except G3 and pass_fail. y contained only pass_fail.
Categorical encoding	Used one-hot encoding with pd.get_dummies(..., drop_first=True), converting text categories into numeric 0/1 columns.
Train/test split	Used 80% training and 20% testing. stratify=y preserved the pass/fail ratio in both sets.

After preprocessing, the encoded feature dataset contained approximately 41 model-ready columns. The target vector y contained the class label only: 1 for pass and 0 for fail.

No normalization or standardization was required for Decision Tree and Random Forest because tree-based models split data by feature thresholds and do not depend on distance or gradient scale like KNN, SVM, or logistic regression can.

4. Exploratory Data Analysis

Exploratory data analysis was used to understand the class distribution and identify important patterns before modeling. The most important pattern was class imbalance: the number of passing students was much higher than the number of failing students.

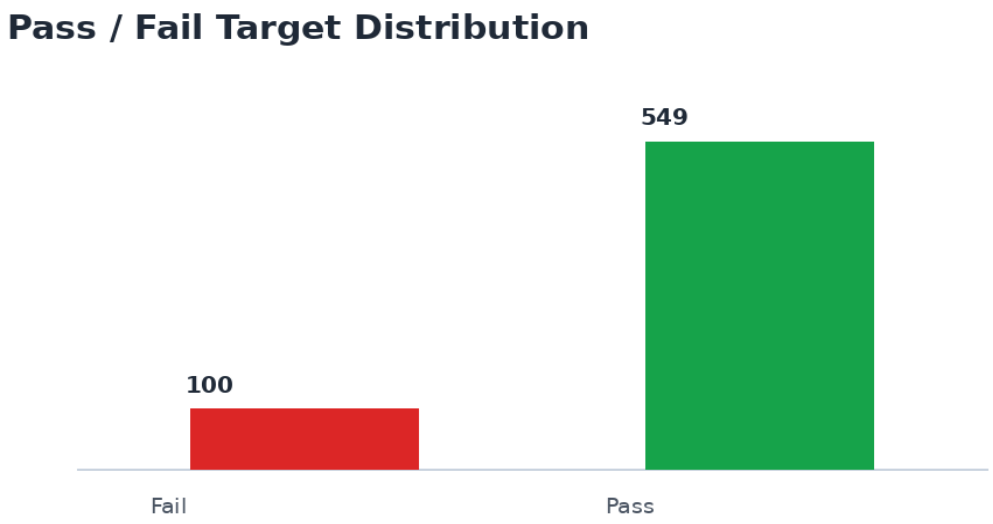

Figure 1. Pass/fail distribution after converting G3 into a binary target.

In the full dataset, approximately 549 students passed and 100 students failed using the rule G3 >= 10. This means a model can achieve high overall accuracy while still being weaker at identifying the smaller Fail class.

The feature importance analysis later confirmed that G2 and G1 were much more influential than most other variables. This is expected because G1 and G2 represent earlier academic performance and are closely related to the final grade G3.

5. Methodology

The methodology followed a supervised machine-learning workflow. Supervised learning was appropriate because the dataset already contained the final result G3, which was converted into the labeled target pass_fail. The models learned patterns from the training data and were then tested on unseen data.

The workflow was: load data, create target, check missing values, remove G3 from input features, encode categorical variables, split the data, train models, predict test labels, and evaluate the predictions.

Model	How it works	Why used	Main settings
Decision Tree	Splits data into branches using feature rules that separate Pass and Fail cases.	Interpretable baseline; easy to explain.	random_state=42, max_depth=5
Random Forest	Builds many decision trees and combines their votes for the final class.	Usually more stable and accurate than one tree.	n_estimators=100, random_state=42, max_depth=5

Decision Tree was limited with max_depth=5. This prevents the tree from becoming too deep and memorizing the training data. Random Forest used 100 trees, meaning the model created many different trees and averaged their class votes.

The dataset was split into training and testing sets using an 80/20 split. Stratification was used so the pass/fail ratio stayed similar in both training and testing sets. The same split was used for both models, making the comparison fair.

The implementation was completed in Google Colab using Python libraries including pandas, scikit-learn, seaborn, and matplotlib.

6. Results and Evaluation

The models were evaluated on the testing data only. This is important because testing data represents unseen examples and gives a more realistic estimate of model performance.

The Decision Tree achieved 88.46% accuracy, while the Random Forest achieved 90.77% accuracy. Therefore, Random Forest was the best-performing model in this experiment.

Method	**Accuracy**	**Main finding**	**Remarks**
Decision Tree	88.46%	Good baseline performance.	Interpretable, but slightly weaker than Random Forest.
Random Forest	90.77%	Best result.	More accurate because it combines many decision trees.
Metric		**Meaning in this project**	
Accuracy		Percentage of all test students classified correctly.	
Precision		When the model predicts a class, how often that prediction is correct.	
Recall		How many real students from a class the model successfully finds.	
F1-score		Balanced score that combines precision and recall.	
Confusion matrix		Shows correct and incorrect predictions for Fail and Pass separately.	

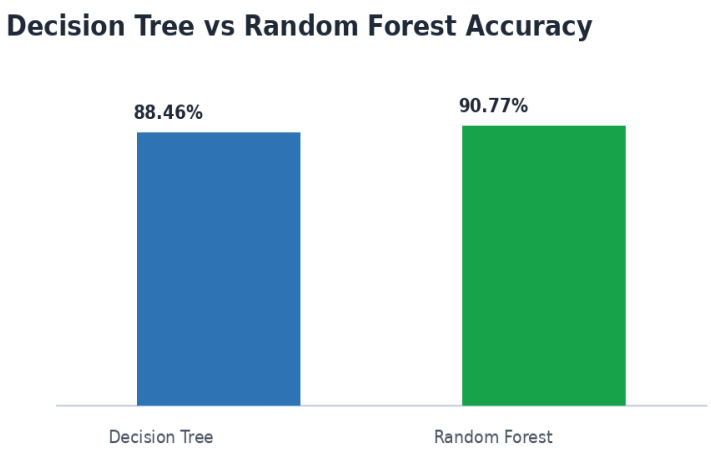

Figure 2. Model accuracy comparison.

The Decision Tree confusion matrix showed 12 correctly predicted fail cases, 103 correctly predicted pass cases, 8 fail students predicted as pass, and 7 pass students predicted as fail. This means the model was stronger for the Pass class than for the Fail class.

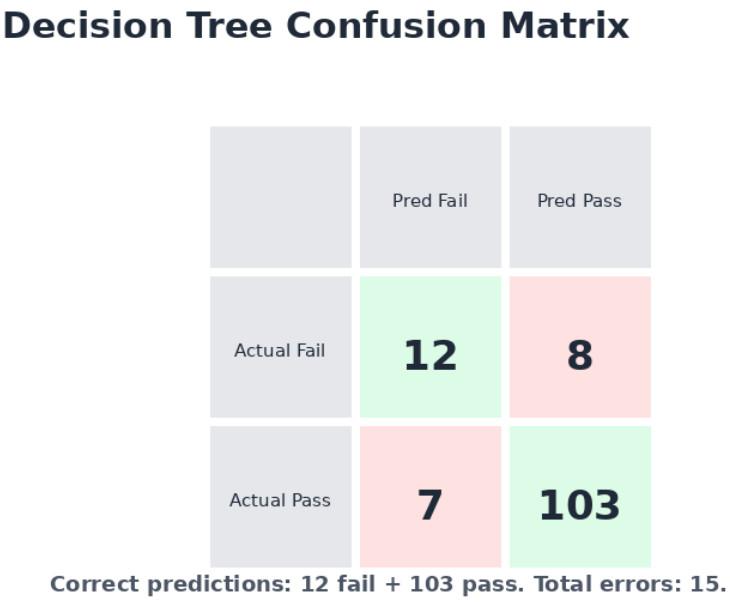

Figure 3. Decision Tree confusion matrix.

Class	Precision	Recall	F1-score	Support
Fail (0)	0.63	0.60	0.62	20
Pass (1)	0.93	0.94	0.93	110
Weighted average	0.88	0.88	0.88	130

For the Fail class, precision was 0.63 and recall was 0.60. For the Pass class, precision was 0.93 and recall was 0.94. The difference shows the effect of class imbalance: the model learned the majority Pass class better than the minority Fail class.

7. Discussion

Random Forest performed better than Decision Tree because it uses an ensemble approach. A single Decision Tree can be sensitive to the exact training data and can overfit if it grows too complex. Random Forest reduces this problem by training many trees and combining their predictions.

The model predicted passing students better than failing students. This happened because the dataset contained many more pass cases than fail cases. In the test set, there were 110 pass students and only 20 fail students, so the model had more examples of passing students to learn from.

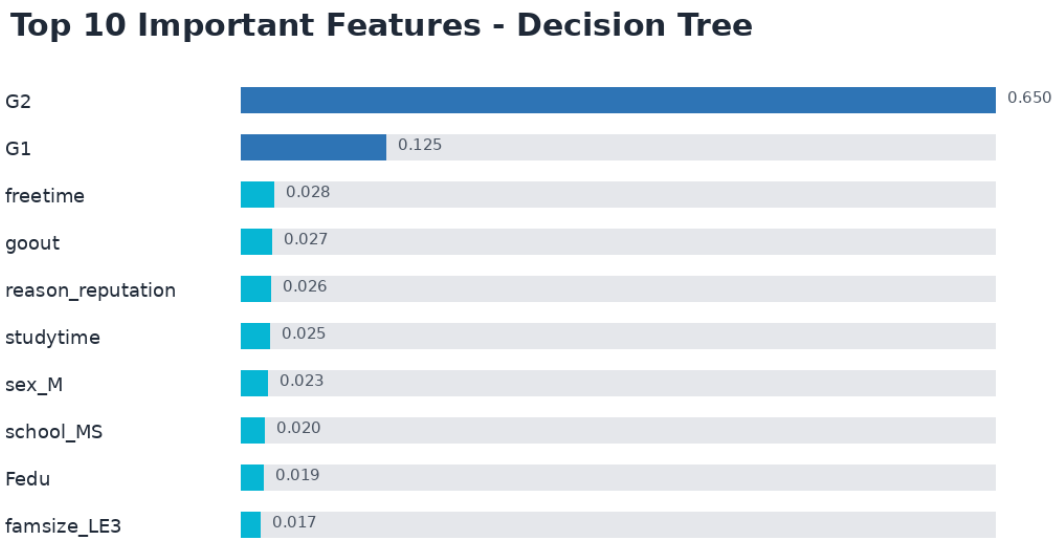

Figure 4. Top 10 important features from the Decision Tree model.

The most important features were G2 and G1. This means previous academic performance was the strongest signal for final performance. Features such as freetime, goout, studytime, school, and family-related variables had smaller effects.

This result is logical: students who performed well in the first and second grading periods were more likely to pass the final grade. However, it also creates an important limitation. Because G1 and G2 are very close to G3, the model is partly predicting final performance from earlier performance in the same subject.

A more challenging version of the project would remove G1 and G2 and try to predict academic risk earlier using only demographic, school, study, absence, and support variables. That version would likely have lower accuracy, but it would be more useful for early intervention.

8. Conclusion and Future Work

This project built a binary classification system for predicting whether a student would pass or fail. The target variable was created from the final grade G3, the dataset was checked for missing values, categorical variables were encoded, and the data was split into training and testing sets.

Two algorithms were tested. Decision Tree was used because it is simple and interpretable. Random Forest was used because it improves the single-tree approach by combining many trees. The Decision Tree achieved 88.46% accuracy and the Random Forest achieved 90.77% accuracy.

Random Forest was selected as the best model because it gave the highest accuracy. The most important predictors were G2 and G1, showing that previous grades are the strongest indicators of final student performance.

Future work could improve the project by balancing the Fail class, testing additional algorithms such as Logistic Regression, Support Vector Machine, or Gradient Boosting, and evaluating models without G1 and G2 to make earlier prediction more realistic.

9. References

[1] P. Cortez, Student Performance Dataset, UCI Machine Learning Repository, 2008. https://archive.ics.uci.edu/dataset/320/student+performance

[2] P. Cortez and A. M. G. Silva, Using Data Mining to Predict Secondary School Student Performance, 2008.

[3] scikit-learn developers, scikit-learn: Machine Learning in Python. https://scikit-learn.org/

[4] pandas development team, pandas Python data analysis library. https://pandas.pydata.org/

[5] Google Colab, cloud-based Python notebook environment. https://colab.research.google.com/

10. Appendix

The full code was written in Google Colab. The main steps included loading the dataset, creating the pass_fail target, preprocessing, training models, evaluating results, and plotting charts.

Implementation summary

Stage	Main code/action
Target	df['pass_fail'] = 1 if G3 >= 10 else 0
Feature set	X = df.drop(['G3', 'pass_fail'], axis=1); y = df['pass_fail']
Encoding	X_encoded = pd.get_dummies(X, drop_first=True)
Split	train_test_split(..., test_size=0.2, random_state=42, stratify=y)
Models	DecisionTreeClassifier(max_depth=5) and RandomForestClassifier(n_estimators=100, max_depth=5)
Evaluation	accuracy_score, confusion_matrix, classification_report, and feature importance

Submission Checklist

Cover page is completed.

Problem statement and objectives are clearly written.

Dataset source and dataset description are included.

Preprocessing steps are explained.

EDA figures/tables are included and interpreted.

Methodology and algorithms are explained.

Results are reported with suitable metrics.

Conclusion and limitations are included.

References are listed.

Code or appendix material is attached if required.